

HOMEWORK 2

Kavya Saranya K V

2025115152

Q1. Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

Code:

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    printf("Original array :\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    for (int i = 0; i < 5; i++) {
        *(ptr + i) = *(ptr + i) * 2;
    }

    printf("Modified array:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    return 0;
}
```

Sample Output:

```
Original array :
10 20 30 40 50
Modified array:
20 40 60 80 100
```

Q2. Calculate string length using pointers.

Code:

```
#include <stdio.h>

int stringLen(char *str) {
    int count = 0;
    while (*(str + count) != '\0') {
        count++;
    }
    return count;
}

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    int i = 0;
    while (str[i] != '\0') {
```

```

        if (str[i] == '\n') {
            str[i] = '\0';
            break;
        }
        i++;
    }

    printf("Length of string = %d\n", strlen(str));

    return 0;
}

```

Sample Output:

```

Enter a string: Hello World
Length of string = 11

```

Q3. Write a function that calculates the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

Code:

```

#include <stdio.h>

void sumAndProduct(int a, int b, int *sum, int *product) {
    *sum = a + b;
    *product = a * b;
}

int main() {
    int a, b, sum, product;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sumAndProduct(a, b, &sum, &product);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}

```

Sample Output:

```

Enter two numbers: 6 4
Sum = 10
Product = 24

```

Q4. Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

Code:

```

#include <stdio.h>
#include <stdlib.h>

struct Item {
    int id;
    float price;
};

int main() {
    int n;
    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item *ptr = (struct Item *)malloc(n * sizeof(struct Item));
    for (int i = 0; i < n; i++) {

```

```

        printf("Enter id and price for item %d: ", i + 1);
        scanf("%d %f", &((ptr + i)->id), &((ptr + i)->price));
    }

    for (int i = 0; i < n; i++) {
        (ptr + i)->price = (ptr + i)->price * 1.10;
    }

    printf("Items after 10%% price increase:\n");
    for (int i = 0; i < n; i++) {
        printf("Id: %d, Price: %.2f\n", (ptr + i)->id, (ptr + i)->price);
    }

    free(ptr);
    return 0;
}

```

Sample Output:

```

Enter number of items: 2
Enter id and price for item 1: 101 50.00
Enter id and price for item 2: 102 75.50
Items after 10% price increase:
Id: 101, Price: 55.00
Id: 102, Price: 83.05

```

Q5. Define two functions with the same signature. Declare a function pointer and use the pointer to call both functions.

Code:

```

#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}

int main() {
    int a = 6, b = 4;
    int (*funcPtr)(int, int);

    funcPtr = add;
    printf("Using function pointer, add: %d\n", funcPtr(a, b));

    funcPtr = multiply;
    printf("Using function pointer, multiply: %d\n", funcPtr(a, b));

    return 0;
}

```

Sample Output:

```

Using function pointer, add: 10
Using function pointer, multiply: 24

```

Q6. Declare an integer, a pointer to it, and a double pointer. Print the values and addresses at each level.

Code:

```

#include <stdio.h>

int main() {
    int num = 25;
    int *ptr = &num;

```

```

int **dptr = &ptr;

printf("Value of num = %d\n", num);
printf("Address of num = %p\n", (void *)&num);

printf("Value of ptr = %p\n", (void *)ptr);
printf("Value pointed to by ptr = %d\n", *ptr);
printf("Address of ptr = %p\n", (void *)&ptr);

printf("Value of dptr = %p\n", (void *)dptr);
printf("Value pointed to by dptr = %p\n", (void *)*dptr);
printf("Value pointed to by *dptr = %d\n", **dptr);
printf("Address of dptr = %p\n", (void *)&dptr);

return 0;
}

```

Sample Output:

```

Value of num = 25
Address of num = 0x7ffea1a52224
Value of ptr = 0x7ffea1a52224
Value pointed to by ptr = 25
Address of ptr = 0x7ffea1a52228
Value of dptr = 0x7ffea1a52228
Value pointed to by dptr = 0x7ffea1a52224
Value pointed to by *dptr = 25
Address of dptr = 0x7ffea1a52230

```

Q7. Sort an array of floating point numbers using selection sort.

Code:

```

#include <stdio.h>

void selectionSort(float arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            float temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    float arr[n];
    printf("Enter %d numbers: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%f", &arr[i]);
    }

    selectionSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%.2f ", arr[i]);
    }
}

```

```

        printf("\n");

    return 0;
}

```

Sample Output:

```

Enter number of elements: 5
Enter 5 numbers: 3.2 1.5 4.8 2.1 0.9
Sorted array: 0.90 1.50 2.10 3.20 4.80

```

Q8. Define a structure Student with id, name and grade. Create an array of student structures and sort them by id, by name, and by grade.

Code:

```

#include <stdio.h>
#include <string.h>

struct Student {
    int id;
    char name[50];
    float grade;
};

void sortById(struct Student arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j].id > arr[j + 1].id) {
                struct Student temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void sortByName(struct Student arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (strcmp(arr[j].name, arr[j + 1].name) > 0) {
                struct Student temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void sortByGrade(struct Student arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j].grade > arr[j + 1].grade) {
                struct Student temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void display(struct Student arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Id: %d, Name: %s, Grade: %.2f\n", arr[i].id, arr[i].name, arr[i].grade);
    }
}

int main() {

```

```

int n;
printf("Enter number of students: ");
scanf("%d", &n);

struct Student students[n];

for (int i = 0; i < n; i++) {
    printf("\nEnter id, name, grade for student %d: ", i + 1);
    scanf("%d %s %f", &students[i].id, students[i].name, &students[i].grade);
}

sortById(students, n);
printf("Sorted by Id:\n");
display(students, n);

sortByName(students, n);
printf("Sorted by Name:\n");
display(students, n);

sortByGrade(students, n);
printf("Sorted by Grade:\n");
display(students, n);

return 0;
}

```

Sample Output:

```

Enter number of students: 3
Enter id, name, grade for student 1: 2 Charlie 88.5
Enter id, name, grade for student 2: 1 Alice 92.0
Enter id, name, grade for student 3: 3 Bob 79.5
Sorted by Id:
Id: 1, Name: Alice, Grade: 92.00
Id: 2, Name: Charlie, Grade: 88.50
Id: 3, Name: Bob, Grade: 79.50
Sorted by Name:
Id: 1, Name: Alice, Grade: 92.00
Id: 3, Name: Bob, Grade: 79.50
Id: 2, Name: Charlie, Grade: 88.50
Sorted by Grade:
Id: 3, Name: Bob, Grade: 79.50
Id: 2, Name: Charlie, Grade: 88.50
Id: 1, Name: Alice, Grade: 92.00

```

Q9. Sort an array of negative integers using radix sort.

Code:

```

#include <stdio.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

void countingSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++) {
        count[(arr[i] / exp) % 10]++;
    }
}

```

```

    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSort(arr, n, exp);
    }
}

int main() {
    int n;

    printf("Enter number of negative integers: ");
    scanf("%d", &n);

    int arr[n], absArr[n];

    printf("Enter %d negative numbers: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        absArr[i] = -arr[i];
    }
    radixSort(absArr, n);

    printf("Sorted negative numbers: ");
    for (int i = n - 1; i >= 0; i--) {
        printf("%d ", -absArr[i]);
    }
    printf("\n");

    return 0;
}

```

Sample Output:

```

Enter number of negative integers: 4
Enter 4 negative numbers: -5 -12 -3 -8
Sorted negative numbers: -12 -8 -5 -3

```

Q10. Copy elements from one array to another using pointer arithmetic.

Code:

```

#include <stdio.h>

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int src[n], dest[n];

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &src[i]);
    }
}

```

```
int *sptr = src;
int *dptr = dest;

for (int i = 0; i < n; i++) {
    *(dptr + i) = *(sptr + i);
}

printf("Copied array: ");
for (int i = 0; i < n; i++) {
    printf("%d ", *(dptr + i));
}
printf("\n");

return 0;
}
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements: 1 2 3 4 5
Copied array: 1 2 3 4 5
```